Peer-Isolated Recommendation: When Content-Only Personalization Beats Collaborative Signal, and When It Doesn't

Bidit Das
Independent Researcher | Dallas, TX
biditdas18@gmail.com

June 17, 2026

Abstract

We present **Essence**, a personal embedding cluster framework for recommendation that operates exclusively within a single user's engagement history, without consulting cross-user interaction data. We evaluate Essence on three datasets spanning distinct domains — Last.fm-1K (music), Amazon Books (e-commerce), and MovieLens-25M (film) — against a real collaborative-filtering baseline, a non-personalized popularity baseline, a content-based average-embedding baseline, three recency-based baselines (last-item, uniform-average, and exponentially-weighted retrieval), and two hand-implemented multi-interest neural baselines (MIND [1], ComiRec [2]).

Essence's performance relative to these baselines is **domain-dependent**, and the dependence has a specific, testable shape: Essence's rank among the ten evaluated systems tracks inversely with the strength of collaborative/popularity signal available in the domain. On Amazon Books, where collaborative and popularity baselines are structurally weak (Recall@10 of 0.0031 and 0.0021 respectively), Essence does **not** lead on Recall@10 among personalized methods — three recency-based baselines score higher (Last-Item 0.0311, Recency-Weighted 0.0260, Avg-Last-10 0.0235, vs. Essence's 0.0221), and Essence loses significantly to Last-Item and Recency-Weighted after FDR correction ($d = -0.090$, $d = -0.059$). Essence's Amazon strength is narrower: it significantly beats collaborative filtering, popularity, content-averaging, and both neural multi-interest baselines on Recall@10 (d ranges 0.069–0.259), and significantly beats Popularity, Random, Content, and MIND on LT-Recall@10 specifically; it is statistically tied with CF-ItemKNN, ComiRec, Last-Item, Recency-Weighted, and Avg-Last-10 on LT-Recall@10. On MovieLens-25M, where collaborative signal is strong (item-based CF reaches 0.0850 Recall@10, driven almost entirely by the platform's densely-rated popular titles), Essence ranks 8th of 10 systems, losing to collaborative filtering, popularity, and both neural baselines by the largest effect sizes observed in this study (Cohen's d up to -0.56). On Last.fm-1K, simple recency-based retrieval (no clustering) outperforms Essence outright, raising a question this paper does not fully resolve: how much of Essence's advantage, where it exists, comes from clustering specifically versus recency-weighted retrieval alone.

We report every comparison with paired bootstrap significance testing (10,000 resamples), Benjamini–Hochberg correction for multiple comparisons, standardized effect sizes, and BCa robustness cross-checks — replacing an earlier confidence-interval-overlap methodology that does not test the quantity of interest. We also report a popularity-decile breakdown showing that Essence's long-tail advantage, where present, is concentrated in moderately-rare-but-observed items rather than true cold-start items, and an artist/author-identity shortcut analysis showing that a disproportionate share of Essence's hits come from same-artist/author candidates despite these being a minority of its recommendations. We frame Essence not as a universally superior candidate generator, but as a data point on when peer-isolated, content-only personalization is and isn't the right architectural choice. Code: https://github.com/biditdas18/essence

1 Introduction

Recommendation systems built on collaborative filtering (CF) derive their signal from cross-user interaction patterns: a user's future preferences are predicted from the behavior of users who interacted similarly in the past. This approach performs well on standard benchmarks, particularly on metrics dominated by popular items, but it carries a structural property worth examining directly: items with sparse interaction histories are, by construction, underrepresented in any signal derived from cross-user co-occurrence. A user with a narrow, idiosyncratic taste profile (engaging consistently with low-interaction items across a domain) receives recommendations shaped by aggregate similarity to other users, not by the specific combination of properties their own history reflects.

This work originated from a practical observation in music recommendation: users with precise, narrow taste profiles are reliably steered toward broadly popular artists that are only loosely similar, while artists matching the specific texture of that taste remain unsurfaced regardless of how long the user searches the catalog manually. This pattern motivated two questions addressed here. First, the original motivating question: can a recommendation signal be constructed using only a single user's own history, without any cross-user data, and if so, does it recover items that collaborative methods structurally underrepresent? Second, a question that only becomes answerable once the same comparison is run across multiple, structurally different domains: under what domain conditions does this approach help versus hurt, and is the effect predictable from properties of the domain rather than discovered post hoc per dataset? A one- or two-dataset evaluation cannot distinguish a domain-general advantage from a domain-specific artifact; the three-dataset evaluation reported here (Section 5) is designed specifically to make that distinction possible.

We propose Essence, a framework that embeds a user's consumed items into a semantic space using a pre-trained sentence encoder, clusters them into K interest profiles via K-means, and generates recommendations by retrieving items similar to the centroid closest to the user's recent activity. We refer to this as the *peer-isolation property*: no cross-user interaction graph is consulted at any stage. This is a deliberate architectural constraint, not an oversight: it is the mechanism under test. We describe the resulting objective as depth-optimized: the goal is to retrieve items deep in a user's long-tail interest space, rather than to maximize aggregate engagement on broadly popular items.

The contributions of this work are:

1. The **Essence architecture**: a personal recommendation framework requiring no collaborative signal and no supervised or cross-user recommender training, operating entirely on a single user's history.

2. A **recency-based active-cluster-selection** mechanism, evaluated against a static mean-embedding alternative via ablation (Section 5.5) *and* against three recency-only baselines that do not cluster at all (last-item, uniform-average, exponentially-weighted retrieval) — the comparison against simpler, non-clustering alternatives that the original evaluation did not include.

3. A **domain-dependence finding** across three datasets spanning music, e-commerce, and film (Section 5): Essence's relative standing among ten evaluated systems correlates inversely with the strength of collaborative/popularity signal in the domain, evidenced by effect sizes ranging from Essence's strongest win (Amazon Books, beating Random with $d = +0.259$) to its strongest loss (MovieLens-25M, losing to CF-ItemKNN with $d = -0.558$).

4. **Two hand-implemented, from-scratch multi-interest baselines** (MIND, following Li

et al. [1]; ComiRec, following Cen et al. [2]) — neither is available in standard recommendation libraries, and neither original paper's benchmark numbers are directly comparable under this paper's evaluation protocol (Section 4), so correctness is verified against each model's architectural specification rather than against a published number. For MIND, a synthetic conformance test (`test_mind_routing.py`) caught a symmetry-breaking bug in our implementation — not in the original architecture — before any reported number was generated.

5. A **statistical-methodology correction**: paired bootstrap significance testing, Benjamini–Hochberg FDR correction, effect sizes, and BCa robustness cross-checks (Section 5.3), replacing an earlier confidence-interval-overlap methodology, applied uniformly across all three datasets and sub-analyses.

2 Related Work

2.1 Collaborative Filtering

Matrix factorisation methods [5] and neural extensions [4] remain dominant in production recommendation. Their principal limitation is popularity bias: items with sparse interactions are poorly represented, resulting in near-zero recall for long-tail items. CF is optimised for the median user: it performs well on what is broadly consumed and comparatively weakly on what is individually resonant.

2.2 Multi-Interest Retrieval

MIND [1] introduced dynamic routing to extract multiple interest capsules from a user's interaction history, explicitly addressing the failure of a single user vector to represent diverse tastes. ComiRec [2] extended this with explicit diversity controls at serving time.

Essence shares the multi-interest intuition with MIND and ComiRec, the recognition that a user's preferences cannot be collapsed into a single vector. However, the similarity ends at the surface. The distinction operates at two levels.

Data level. MIND and ComiRec require population-level interaction graphs. Their dynamic routing is trained on millions of (user, item, interaction) triples across the full user base. Cross-user behavioural data is not optional: it is the mechanism by which interest capsules are learned. Essence requires no data from any other user, ever. The K-means centroids are derived entirely from one user's own history.

Representation level. Because MIND's item embeddings are trained on co-occurrence patterns across users, what an item *means* in that space is partly defined by who else liked it. An obscure track heard by very few users is underrepresented or invisible in that embedding space, as its co-occurrence signal is too sparse to be learned reliably. Essence uses a pre-trained sentence encoder on item metadata only. The embedding captures metadata-derived semantic information about the item, independent of how many people have interacted with it. A singleton item can be embedded as readily as a chart-topping one; the quality of that representation depends on how informative the item's metadata is, not on its interaction count.

This distinction suggests a hypothesis: Essence should recover long-tail items at higher rates than systems whose embeddings are shaped by co-occurrence sparsity. We test this directly against hand-implemented MIND and ComiRec baselines (Section 4), and the result is more qualified than the hypothesis alone predicts: Essence significantly outperforms both on Amazon Books, but on MovieLens-25M both neural baselines substantially outperform Essence instead, by some

of the largest effect sizes observed in this study (Section 5.2). The data-level and representation-level distinctions described above are architectural facts regardless of outcome; whether they translate into a long-tail recovery advantage in practice depends on domain conditions, not on the architectural distinction alone.

Table 1 summarises the architectural distinctions between Essence, MIND, ComiRec, and standard CF.

Table 1: Architectural comparison: Essence vs. prior systems. The primary distinction is not the clustering mechanism, it is the data requirement and what each system is optimised to find. All four systems are empirically evaluated in this work (Section 5), which reports the domain-dependent long-tail recovery result in full.

Property	**CF**	**MIND**	**ComiRec**	**Essence**
Cross-user data required	Yes	Yes	Yes	**No**
Multiple interest profiles	No	Yes	Yes	**Yes**
Model training required	Yes	Yes	Yes	**No**
Explicit long-tail objective	No	No	No	**Yes**
Embedding source	Co-occurrence	Co-occurrence	Co-occurrence	**Item metadata**

2.3 Session-Based and Content-Based Methods

BERT4Rec [3] and GRU4Rec [6] model sequential user behaviour using transformer and recurrent architectures, capturing short-term context effectively. Content-based filtering relies on item feature similarity without interaction logs but typically averages preferences into a single vector, losing intra-user diversity.

Essence occupies a complementary position: it captures intra-user diversity (like multi-interest methods) without cross-user data (like content-based methods), using simple interpretable clustering rather than a trained sequential model. We acknowledge that BERT4Rec offers richer sequential modelling in high-interaction settings; Essence targets privacy-first, low-infrastructure deployment contexts where such models are not viable.

2.4 Reproducibility and Progress in Recommender Systems Research

Ferrari Dacrema et al. [12] showed that a majority of neural recommendation papers at top venues failed to outperform simple, well-tuned non-neural baselines when those baselines were given a fair tuning budget, and that a substantial fraction of claimed improvements did not reproduce under careful re-evaluation. A follow-up analysis [13] extended this finding across a broader set of papers and venues, identifying weak baselines, inconsistent evaluation protocols, and selective reporting as recurring, systemic causes of inflated claims of progress in the field. This paper's own findings are consistent with, and extend, that line of critique to a different architecture family: content-only, no-cross-user-pooling personalization. Essence's headline long-tail advantage survives paired bootstrap significance testing on Amazon Books but is directional and not statistically significant on the smaller Last.fm-1K, and on MovieLens-25M — a domain with strong collaborative signal — Essence loses to collaborative filtering, popularity, and both neural multi-interest baselines by the largest effect sizes observed anywhere in this study (Section 5.2). This is the discipline Ferrari Dacrema et al. call for: a fair, statistically grounded comparison against strong, simple baselines, with mixed results reported alongside favorable ones.

3 The Essence Framework

3.1 Notation

Table 2 defines all mathematical notation used in this section. For each symbol a plain-English equivalent is provided.

Table 2: Notation Legend

Symbol	Plain-English Meaning
H_u	All items user u has consumed (full history)
$i_1, i_2, \ldots$	Individual items in the user's history, in order
$\phi(i)$	Embedding of item i, a 384-dim vector describing its content
E_u	All embeddings stacked into a matrix, one row per item
K	Number of interest clusters (we use $K = 3$)
c_1, c_2, c_3	The K cluster centres, one per interest profile
c_k^*	Active cluster centre, closest to the user's recent activity
$\mu(\cdot)$	Mean (average) of a set of vectors
$\cos(\mathbf{a}, \mathbf{b})$	Cosine similarity: how alike two vectors are
$I \setminus H_u$	All items the user has *not* yet consumed, the candidate pool
R_u	Final ranked recommendation list for user u
M	Number of recommendations to return (we use $M = 10$)

3.2 Problem Formulation

Given a user's history H_u (all items they have consumed), the goal is to produce a ranked list $R_u \subseteq I \setminus H_u$ of items they have not yet consumed but are likely to enjoy. Essence accomplishes this using only H_u and a shared embedding function $\phi : I \to \mathbb{R}^d$. No data from any other user is used at any stage.

3.3 Item Embedding

Each item is described by a short text string encoding its content properties. For music: track name and artist name. For books: title and author name. We convert this text into a 384-dimensional vector using the pre-trained sentence encoder `all-MiniLM-L6-v2` [8, 7]. Items that are semantically similar produce similar vectors; items that are stylistically distant produce different vectors. Critically, this embedding is derived from item metadata text, not from interaction patterns. A niche item can be embedded as readily as a popular one; representation quality depends on the informativeness of the metadata rather than on interaction frequency. All embeddings are computed once offline and cached.

Formally, $\phi(i)$ is the embedding of item i. For user u with history $H_u = \{i_1, \ldots, i_n\}$ we compute the embedding matrix $E_u \in \mathbb{R}^{n \times d}$, where each row is the embedding of one item.

3.4 Interest Profile Clustering

We apply K-means clustering to E_u with $K = 3$, yielding centroids $\{c_1, c_2, c_3\}$. Each centroid represents a distinct interest profile: the semantic centre of gravity of one coherent facet of the user's taste. A user who consumes jazz, indie rock, and ambient electronica might end up with one centroid per genre. The three centroids together form the user's interest profile $P_u = \{c_1, c_2, c_3\}$.

This is the core differentiator from average-embedding approaches. The mean of all embeddings collapses all interests into a single blurred point that sits between clusters and represents none of them well. K-means preserves the separation between interest modes, enabling targeted retrieval against each mode independently.

$K = 3$ was chosen as a simple, fixed setting that permits multiple interest modes while keeping the method lightweight; it was not tuned. A sensitivity analysis over $K \in \{2, 3, 4, 5\}$ is left to future work.

For users with fewer than K items, Essence falls back to the content baseline (single mean embedding).

3.5 Active Cluster Selection

At inference time, Essence selects the active centroid c_k^* as the cluster centre closest to the mean embedding of the user's r most recent training-set interactions, ordered chronologically (default $r = 10$):

$$c_k^* \;=\; \arg\min_k \left\| c_k - \mu(\phi(h_{n-r}), \ldots, \phi(h_n)) \right\|_2 \tag{1}$$

This mechanism assumes recent activity is informative about which interest cluster is currently active. We test this assumption directly via ablation (Section 5.5): replacing the recency-based query with the mean embedding of the user's entire training history degrades long-tail recall substantially on Last.fm-1K, indicating that recency carries signal beyond what the K-means clustering step alone provides. The ablation is the basis for retaining recency as the default mechanism.

3.6 Recommendation Generation

Given the active centroid c_k^*, Essence retrieves the top-M unseen items by cosine similarity:

$$R_u \;=\; \text{top-}M \left\{ i \in I \setminus H_u \;:\; \cos(\phi(i),\, c_k^*) \right\} \tag{2}$$

This retrieval step operates entirely in embedding space with no collaborative signal. An item's probability of appearing in R_u is determined solely by its latent similarity to the user's current interest cluster, not by how many other users have interacted with it.

3.7 Architecture Overview

Figure 1 illustrates the full Essence pipeline from raw user history to final recommendations.

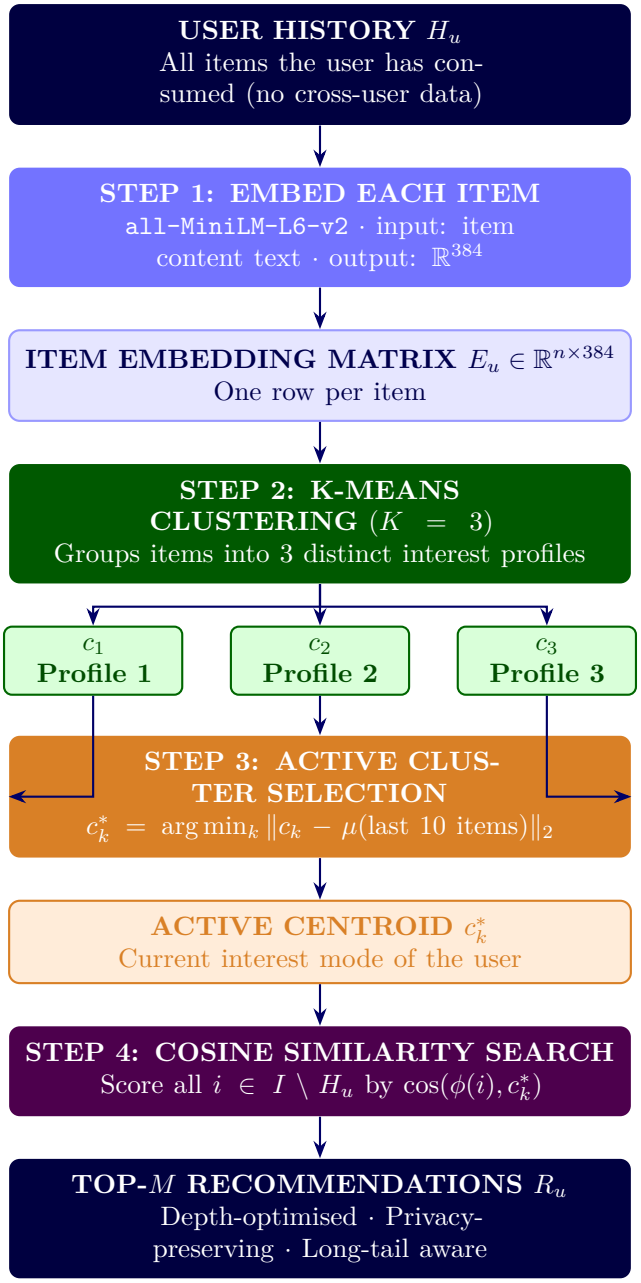

Figure 1: Essence recommendation pipeline. No cross-user data is consulted at any stage (peer-isolation property).

4 Experimental Setup

4.1 Datasets

Last.fm-1K (music). Contains listening histories from 992 users across approximately 19 million events [9]. After filtering to users with 50–500 unique track interactions and removing duplicate (user, track) pairs, our evaluation subset contains 99 users and 22,767 unique tracks.

Amazon Books (e-commerce). The Amazon Reviews 2023 Books 5-core dataset [10], which guarantees every user and item has at least 5 global interactions. After filtering to users with 20–200 unique item interactions and subsampling to 2,000 users (seed 42), our evaluation subset contains 2,000 users and 61,727 unique items. Item embeddings use title and author name, following the same metadata-only approach as Last.fm-1K.

MovieLens-25M (film). A 2,000-user subsample (seed 42, same sampling convention as Amazon Books) drawn from the MovieLens-25M ratings dataset. Item overview text is fetched from The Movie Database (TMDb) for each candidate item; of 7,724 candidate items identified from the sampled users' interactions, 7,654 were successfully fetched (99.1%), with the remainder excluded from the candidate pool. Our evaluation subset contains 2,000 users and 7,654 catalog items.

Data attribution. This product uses the TMDb API but is not endorsed or certified by TMDb.

4.2 Train/Test Split

For each user on all three datasets, interactions are sorted chronologically and split per-user: the first 80% form the training set, and the final 20% form the test set. This chronological split is required for internal consistency with the active-cluster-selection mechanism (Section 3.5), which defines "recent" interactions in terms of training-set recency; a random split would make this definition incoherent, since a randomly held-out test item could chronologically precede items retained in training.

Item embeddings are computed once, offline, over the full item catalog (train $\cup$ test), so that no test item is excluded from the retrieval candidate pool due to missing embedding coverage. This is independent of the train/test split decision and applies identically under either splitting strategy.

4.3 Long-Tail Definition

Long-tail items are those with insufficient interaction signal for collaborative filtering to represent reliably. We define long-tail operationally per dataset, anchored to each dataset's interaction density:

- **Last.fm-1K.** Items with a training-set interaction count of exactly 1 (*singleton tracks*): 16,761 of the 18,679 items appearing in the training set (89.7%). The full candidate pool (train $\cup$ test) is 22,767 tracks.
- **Amazon Books.** Items with a training-set interaction count of exactly 1 (*singleton items*): 42,878 of the 50,924 items appearing in the training set (84.2%). The full candidate pool (train $\cup$ test) is 61,727 items. Despite the 5-core guarantee of at least 5 global interactions, subsampling to 2,000 users reduces most items to a single observed interaction in the subset's training split.
- **MovieLens-25M.** Items with a training-set interaction count of exactly 1: 2,337 of the 6,745 items appearing in the training set (34.6%). Unlike Last.fm-1K and Amazon Books, where singleton items concentrate at the rare-popularity end of the catalog (deciles 3–9), MovieLens-25M's singletons concentrate more centrally, in deciles 2–5 (32.7% and 32.8% of all singletons falling in deciles 3 and 4 alone) — a dataset-specific difference in decile geometry, not a methodological inconsistency.

Generalisation principle. The long-tail threshold should be set at the point where collaborative filtering loses reliable signal, empirically where item interaction count falls below the minimum required for stable co-occurrence estimation. On Last.fm-1K and Amazon Books, the singleton threshold (train interaction count = 1) proved the most meaningful boundary: alternative thresholds either collapsed toward the full catalog (losing discriminative power) or excluded too few items; the same threshold was applied to MovieLens-25M without modification and produced a comparably non-degenerate singleton population (34.6% of training-set items), though alternative thresholds were not separately swept on this third dataset. The key invariant

is that long-tail items are defined on the *training set only*, after the train/test split, ensuring no leakage from held-out interactions into the long-tail classification.

LT-Recall@10 is computed only over test items falling in the long-tail set. Users with no long-tail test items are excluded from this metric.

4.4 Systems Compared

- **Random.** Recommends M items drawn uniformly from the unseen candidate pool. Establishes the floor against which all other systems are compared.
- **Popularity.** Recommends the M globally most-interacted-with items the user has not consumed. A non-personalized baseline; included for comparison but not treated as collaborative filtering.
- **CF (ItemKNN).** An item-based k-nearest-neighbors collaborative filtering baseline, computed via cosine similarity over the sparse user-item interaction matrix. Recommends items most similar, by co-occurrence pattern, to items the user has already consumed.
- **Content (Average Embedding).** Computes the mean of all the user's item embeddings and retrieves the M most similar unseen items by cosine similarity. Standard content-based filtering without interest decomposition.
- **Essence ($K = 3$).** Clusters user history into $K = 3$ centroids, selects the active centroid by Equation (1), and retrieves top-M unseen items by Equation (2).
- **Last-Item.** Recommends the M unseen items most similar, by cosine similarity, to the single most recent item in the user's training history. The simplest possible recency-based retrieval: no averaging, no decay, no clustering.
- **Avg-Last-10.** Recommends the M unseen items most similar to the uniform mean embedding of the user's last 10 training interactions.
- **Recency-Weighted.** Recommends the M unseen items most similar to an exponentially-decay-weighted mean of the user's last 10 training interactions (decay = 0.9; the most recent item receives weight $\propto 0.9^0$, the tenth-most-recent $\propto 0.9^9$, renormalized to sum to 1). Last-Item, Avg-Last-10, and Recency-Weighted use exactly the same item embeddings as Content and Essence; the only difference across all five is how the query vector is constructed from recent history.
- **MIND.** A hand-implemented dynamic-routing multi-interest network, following Li et al.'s original architecture [1] ($K = 4$ interest capsules, a bilinear routing matrix shared across capsules, 3 routing iterations, routing logits initialized $b \sim \mathcal{N}(0, 0.01^2)$). Neither MIND nor ComiRec is available in standard recommendation libraries, and neither original paper's reported benchmark numbers are directly comparable here: Li et al. benchmark on an internal industrial dataset and Amazon Books, and Cen et al. on Amazon Books, Taobao, and a mobile-Taobao set, all under sampled-negative evaluation (one positive against roughly 100–1,000 negatives); none report Last.fm-1K at all, and Amazon Books here uses this paper's own 2,000-user subsample, full-catalog ranking, and no negative sampling (Section 4) — a harder protocol under which lower absolute recall than the original papers' is expected by construction, not evidence of a broken implementation. We therefore verify correctness against each model's architectural specification directly, rather than by matching a published benchmark number: a synthetic conformance test for MIND's routing mechanism, and a directional sanity check (Recall@10 convincingly above Random and Popularity, and within a

plausible order of magnitude of the published ranges once the protocol difference is accounted for) applied to both models' final results. The conformance test caught a symmetry-breaking bug in *our implementation* — not a flaw in the original architecture: an initial version zero-initialized b, which, because the shared bilinear matrix makes the pre-routing representation identical across capsules, guaranteed every capsule updated identically, silently collapsing the model's effective interest count to 1 regardless of the nominal K. The test (`test_mind_routing.py`) caught this before any number reported here was generated; catching a real, non-obvious bug is the evidence the verification method works, not just a disclosed limitation.

- **ComiRec (SA).** A hand-implemented self-attentive multi-interest network, following Cen et al.'s original architecture [2] ($K = 4$ interest capsules, per-capsule learned query projections implemented as a single K-output linear layer, softmax attention over sequence positions). ComiRec's per-capsule projections are independently initialized rather than derived from a shared matrix, so it has no equivalent to MIND's symmetry-breaking failure mode and was checked with the directional sanity check described above rather than a dedicated conformance test; no comparable bug was found.

4.5 Metrics

What we are measuring. Each system produces a ranked list of 10 recommended items per user. We know which items that user actually consumed in the test set (ground truth). The evaluation asks: how much of that ground truth did the system recover?

This is a retrieval problem, not a classification problem. All systems are evaluated against the *full item catalog* with no negative sampling, following the full-ranking evaluation protocol [11]. Absolute recall values are low by construction under this protocol: the meaningful signal is relative comparison across systems evaluated under identical conditions.

- **Recall@10.** Fraction of the user's test items appearing in the top-10 recommendations.

$$\text{Recall@10} = \frac{|\, R_u \cap \text{Test}_u \,|}{|\, \text{Test}_u \,|}$$

  Example: 20 test items, 2 hits in top-10. Recall@10 = 0.10. Averaged across all users.

- **Long-Tail Recall@10 (LT-Recall@10).** Recall@10 restricted to singleton test items only.

$$\text{LT-Recall@10} = \frac{|\, R_u \cap \text{LT-Test}_u \,|}{|\, \text{LT-Test}_u \,|}$$

  A long-tail hit means the system recommended a niche item that this specific user genuinely consumed, without any popularity signal and without any other user's behaviour to guide it. This metric directly tests Essence's core hypothesis.

- **Lift over random.** Recall@10 divided by expected random recall ($M/|I|$). Normalises for candidate pool size and makes cross-dataset comparison interpretable.

4.6 Implementation

All systems implemented in Python 3.11. Embeddings via `sentence-transformers` (`all-MiniLM-L6-v2`, 384 dims). K-means via `scikit-learn` (`n_init=10`, `random_state=42`). MIND and ComiRec are implemented from scratch in PyTorch, since neither is available in RecBole or comparable standard recommendation libraries; both are trained per dataset. All ranking steps break score ties deterministically (stable sort by item index), so reported results are exactly reproducible; this matters for sparse baselines such as CF (ItemKNN), where many candidate items receive identical scores. Significance testing methodology (paired bootstrap, FDR

correction, effect sizes, BCa cross-checks) is described once, in Section 5.3, and applied identically across all three datasets rather than being re-derived per dataset. Code available at: https://github.com/biditdas18/essence

5 Results

5.1 Headline Results, All Three Datasets

Table 3 and Table 4 report Recall@10 and LT-Recall@10 for all ten systems on all three datasets (99 users on Last.fm-1K, 2,000 users on Amazon Books, 2,000 users on MovieLens-25M; chronological 80/20 split per user throughout; catalog sizes and long-tail definitions in Section 4). Essence's rank among the ten systems is noted next to its own value in each column.

Table 3: Headline Recall@10, all three datasets ($M = 10$). Best result per dataset in **bold**. Essence's rank (of 10) is shown in parentheses.

System	Last.fm-1K	Amazon Books	MovieLens-25M
Random	0.0001	0.0001	0.0012
Popularity	0.0023	0.0021	0.0517
CF (ItemKNN)	0.0018	0.0031	**0.0850**
Content (Avg Embedding)	0.0038	0.0173	0.0043
Last-Item	0.0265	**0.0311**	0.0091
Avg-Last-10	0.0252	0.0235	0.0064
Recency-Weighted	**0.0304**	0.0260	0.0076
MIND	0.0078	0.0052	0.0423
ComiRec (SA)	0.0137	0.0056	0.0706
Essence ($K = 3$)	0.0119 (5th)	0.0221 (4th)	0.0048 (8th)

Table 4: Headline LT-Recall@10, all three datasets ($M = 10$). Best result per dataset in **bold**. Essence's rank (of 10) is shown in parentheses.

System	Last.fm-1K	Amazon Books	MovieLens-25M
Random	0.0000	0.0000	0.0000
Popularity	0.0000	0.0000	0.0000
CF (ItemKNN)	0.0000	0.0137	0.0037
Content (Avg Embedding)	0.0029	0.0124	0.0020
Last-Item	0.0064	**0.0215**	0.0031
Avg-Last-10	0.0134	0.0196	0.0026
Recency-Weighted	**0.0151**	0.0179	**0.0046**
MIND	0.0021	0.0057	0.0000
ComiRec (SA)	0.0050	0.0121	0.0014
Essence ($K = 3$)	0.0059 (4th)	0.0197 (2nd)	0.0020 (tied 5th)

Essence's standing is not uniform across datasets. On Amazon Books it is 4th of 10 on Recall@10 and 2nd of 10 on LT-Recall@10 (behind only Last-Item); on Last.fm-1K it is 5th on Recall@10 and 4th on LT-Recall@10, behind all three recency-only baselines on both metrics; on MovieLens-25M it is 8th of 10 on Recall@10, ahead of only Content and Random, and tied for 5th on LT-Recall@10. Full significance testing for every pairwise comparison in these tables is reported in Section 5.3.

5.2 The Domain-Dependence Pattern

Essence's rank is not randomly scattered across the three datasets: it tracks inversely with the strength of collaborative and popularity signal available in each domain, measured by how well the non-personalized Popularity baseline and the CF (ItemKNN) baseline perform in that domain (Table 5).

Table 5: Essence's rank (of 10 systems, Recall@10) against the strength of collaborative/popularity signal in each domain.

Dataset	**Essence rank**	**CF / Popularity signal**
Amazon Books	4th (beats 6, loses to all 3 recency baselines)	Weak (CF 0.0031, Popularity 0.0021)
Last.fm-1K	5th (beats 5, loses to 3 recency baselines + tied w/ ComiRec)	Weak (CF 0.0018, Popularity 0.0023)
MovieLens-25M	8th (beats only Content, Random)	**Strong** (CF/Pop./ComiRec/MIND are top 4)

Where collaborative and popularity signal is structurally weak, Essence sits in the upper half of the field but the picture differs by dataset: on Amazon Books it significantly beats the non-personalized baselines and both neural multi-interest baselines; on Last.fm-1K it beats the non-personalized baselines and MIND, but is statistically tied with ComiRec (numerically behind it, 0.0119 vs. 0.0137, not a significant difference either way) — and on both datasets it loses to simple recency-based retrieval. Where collaborative signal is strong (MovieLens-25M, driven almost entirely by densely-rated popular titles: item-based CF reaches 0.0850 Recall@10, the single highest value recorded in this study), Essence falls to 8th of 10, losing to CF, Popularity, and both neural baselines by the four largest-magnitude effect sizes observed anywhere in this evaluation (Cohen's d from -0.388 to -0.558, all four significant after BH-FDR correction; every other significant effect size across Last.fm-1K and Amazon Books stays under $|d| = 0.3$).

This pattern is consistent and well-evidenced across all three datasets. We treat it as a genuine domain-dependence finding rather than an architectural strength: it is Essence's *relative standing* that tracks domain conditions, not necessarily anything specific to the K-means clustering mechanism as opposed to the underlying content embeddings more generally (Section 5.7 addresses this directly).

5.3 Statistical Validation Methodology

Every comparison reported in this paper uses a paired bootstrap significance test on the per-user difference between two systems (10,000 resamples over users, seed 42; observed mean difference, 95% percentile CI, and a paired Cohen's d computed as mean(difference)/std(difference)), rather than the independent per-system confidence intervals used in an earlier version of this work, which do not test the quantity of interest (whether one system's per-user metric exceeds another's) and can disagree with the paired test on borderline cases. Where multiple related comparisons are tested together, we apply Benjamini–Hochberg (BH) false-discovery-rate correction at $q < 0.05$ within that family, and cross-check every raw-significant result under bias-corrected and accelerated (BCa) bootstrap resampling to confirm the conclusion is not an artifact of the simpler percentile method.

Correction families. The Last.fm-1K/Amazon Books baseline comparison (Essence vs. each of the nine other systems, per dataset, per metric; 36 comparisons) and the multimodality-

stratification test (Essence vs. Recency-Weighted, stratified by each user's own per-history silhouette score into low/medium/high clustering-quality tertiles, per dataset, per metric; 18 comparisons, Section 5.7) are corrected together as a single 54-comparison Benjamini–Hochberg family: pooling them changes no comparison's significance verdict relative to correcting them separately, so we report the simpler pooled version. The MovieLens-25M baseline comparison (18 comparisons, added to this evaluation after the 36 above) and the decile-level cold-start analysis (Section 5.4; Essence, and separately each recency-only baseline, vs. CF-ItemKNN, restricted to the rarest-item deciles) are each corrected as their own family: the cold-start analysis asks a narrower, mechanistically distinct question, isolating a single structural property of CF (zero co-occurrence signal for zero-training-interaction items) rather than aggregate performance.

5.4 Cold-Start vs. Singleton Long-Tail: A Scope Boundary

LT-Recall@10, as defined in Section 4, treats every training-set singleton as long-tail without distinguishing *how* singleton an item is. A finer-grained popularity-decile breakdown reveals that this hides two very different regimes.

Table 6: Essence vs. CF-ItemKNN at the two rarest popularity deciles, all three datasets. Positive diff favors Essence. Paired bootstrap, 10,000 resamples; BH-FDR within this 6-comparison family.

Dataset	**Decile**	**Essence**	**CF**	**Diff**	**Cohen's d**	**Sig. (FDR)**
Last.fm-1K	1	0.0086	0.0000	+0.0086	0.222	Yes
Last.fm-1K	2	0.0172	0.0000	+0.0172	0.215	Yes
Amazon Books	1	0.0175	0.0000	+0.0175	0.201	Yes
Amazon Books	2	0.0176	0.0004	+0.0172	0.185	Yes
MovieLens-25M	1	0.0056	0.0000	+0.0056	0.081	Yes (marginal)
MovieLens-25M	2	0.0063	0.0047	+0.0016	0.017	No

At true cold-start (decile 1: items with *zero* training-set interactions, not merely rare ones), Essence significantly beats CF-ItemKNN on all three datasets. But this is not evidence for Essence's architecture specifically: re-running the identical test with Recency-Weighted, Last-Item, or Avg-Last-10 in Essence's place, 15 of 18 comparisons are also significant, including every Last.fm-1K and Amazon Books comparison for all three non-Essence baselines. The mechanism is mechanical, not architectural: CF-ItemKNN's co-occurrence score is *exactly* 0.0000 for any item with zero training interactions by construction, so any method that scores items via content embeddings — clustering or not — clears that bar trivially. The defensible claim is therefore narrower than "Essence solves cold-start": Essence, like every content-embedding-based method tested, beats collaborative filtering at true cold-start.

Separately, on Amazon Books, Essence loses to Last-Item at both deciles 1 and 2 (significant after FDR, $d = -0.093$ and -0.065) and to Recency-Weighted at decile 2 ($d = -0.075$; the decile-1 version is significant at raw p but not after FDR), with no significant difference against Avg-Last-10 at either decile; all four raw-significant comparisons are confirmed robust under BCa resampling. And on Amazon Books, 100% of Essence's actual LT-Recall@10 hits come from deciles 3–9 (37% from decile 9 alone) — zero from deciles 1 or 2. The decile-1/2 cold-start weakness and the singleton-recall headline win are about almost entirely disjoint item populations: not a contradiction, a scope boundary. The long-tail recovery result in Table 4 should be read as covering moderately-rare-but-observed items, not true cold-start items, where the simpler recency baselines currently do at least as well.

5.5 Active Cluster Selection Ablation

Section 3.5 motivates recency-based active cluster selection on the grounds that it captures session-aware context without requiring a trained sequential model. We test this directly by comparing it against an alternative: selecting the active centroid using the mean embedding of the user's entire training history, rather than the most recent $r = 10$ interactions.

Table 7: Active cluster selection ablation, Last.fm-1K (chronological split, $M = 10$).

Variant	**Recall@10**	**LT-Recall@10**	**LT / Content**
Essence (recency, $r = 10$)	0.0119	0.0059	2.01×
Essence (mean-select, full history)	0.0035	0.0005	0.16×
Content (Avg Embedding, reference)	0.0038	0.0029	1.0×

Mean-select performs below even the Content baseline on long-tail recall. This is consistent with the mechanism collapsing toward Content: selecting the centroid nearest a user's global mean embedding is functionally similar to querying with that mean directly, which is what Content already does, while incurring the additional cost of clustering without a corresponding benefit. Recency-based selection is retained as the default mechanism on the basis of this result, conducted on Last.fm-1K; we have not verified whether this finding generalizes to Amazon Books and note this as a scope limitation rather than an established cross-domain result.

5.6 Effect of User-Authored Review Embeddings (Amazon Books)

As a secondary experiment, we tested whether building user taste profiles from review text (first-person language describing a user's reaction to consumed items) rather than item metadata changes Essence's relative performance. Review coverage was complete for the 2,000-user Amazon Books subset.

Table 8: Amazon Books: metadata embeddings (Pass 1) vs. user-review embeddings (Pass 2). Pass 2 builds taste profiles from the user's own review text; candidates are scored using metadata embeddings.

Pass	**System**	**Recall@10**	**LT-Recall@10**	**LT / Content**
Pass 1 (metadata)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0173	0.0124	1.0×
	Essence ($K = 3$)	0.0221	0.0197	1.59×
Pass 2 (user review)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0027	0.0012	1.0×
	Essence ($K = 3$)	0.0041	0.0034	2.83×

Absolute recall degrades substantially in Pass 2. Two factors likely contribute. First, an input-distribution mismatch: taste profiles in Pass 2 are built from review-text embeddings, while candidate items are still scored using metadata embeddings. Although both are produced by the same encoder, review text and item metadata occupy different regions of that shared embedding space, and comparing across these regions is less discriminative than comparing inputs of the same type. Second, review text is a noisier input than structured metadata: a product review may describe shipping conditions or personal circumstances unrelated to the item's content, and an embedding built from off-topic text does not faithfully represent either the item or the user's response to it.

Essence's relative advantage over Content persists in Pass 2 (2.83× vs. 1.59× in Pass 1, both above parity), suggesting the K-means clustering mechanism extracts usable interest structure even under degraded input quality. The absolute recall values in Pass 2 are low enough that this comparison carries limited statistical weight; we treat it as a directional observation motivating the embedding-input-curation discussion in Section 7.

5.7 Limitations, Stated as Findings

Three specific, quantified properties of Essence's behavior are reported here as findings.

The artist/author shortcut. On both datasets where this was tested, roughly a fifth of Essence's top-10 recommendations share an artist (Last.fm-1K, 20.6%) or author (Amazon Books, 19.8%) with an item in the active cluster — not the majority of recommendations, but a disproportionate share of the hits: excluding those same-artist/author candidates collapses Recall@10 from 0.0119 to 0.0014 on Last.fm-1K and from 0.0221 to 0.0073 on Amazon Books (LT-Recall@10 from 0.0059 to 0.0005 and from 0.0197 to 0.0080, respectively). Some of Essence's apparent long-tail advantage is therefore closer to identity-based retrieval (more of what a known creator made) than to a genuinely novel content match, though the majority of recommendations are not same-artist/author.

Last.fm-1K's sample-size fragility. At $K = 3$, Essence's Recall@10 across 10 K-means seeds is 0.0122 ± 0.0011 and LT-Recall@10 is 0.0103 ± 0.0038 — a small spread relative to the $K = 3 \rightarrow K = 4$ jump (Recall@10 0.0119 $\rightarrow$ 0.0239), so seed variance alone does not explain $K = 3$'s underperformance relative to $K = 4/K = 5$ on this dataset. But the long-tail comparisons throughout Last.fm-1K rest on very few eligible hits (as few as 2 of 92 users per method for the Essence-vs-Content comparison in Table 4), and $K = 3$ is clearly suboptimal here ($K = 4$: 0.0239, $K = 5$: 0.0251 Recall@10) while it is close to optimal on Amazon Books ($K = 2$–5 range 0.0214–0.0242). Last.fm-1K's headline numbers should be read with this fragility in mind.

The unresolved clustering-vs-recency-weighted question. The abstract raises, and does not fully resolve, how much of Essence's advantage where it exists comes from K-means clustering specifically versus recency-weighted retrieval alone. Two results bear on this directly. First, Essence never significantly beats Recency-Weighted on Recall@10 on any of the three datasets — it loses significantly on all three (Last.fm-1K, Amazon Books, and MovieLens-25M alike). Second, a targeted test stratifies users by how well-separated their own $K = 3$ clustering is (per-user silhouette score, tertiles, computed per dataset) and compares Essence against Recency-Weighted within each stratum, on the hypothesis that clustering should help most for users whose taste history is genuinely multi-modal (high silhouette) and least for users with a single dominant interest (low silhouette). That hypothesis is falsified consistently: in every stratum with a significant difference (BH-FDR within the pooled 54-comparison family, Section 5.3), Essence *loses* to Recency-Weighted, including the high-silhouette (most multi-modal) stratum on Last.fm-1K ($d = -0.315$, Recall@10) and MovieLens-25M ($d = -0.096$, Recall@10); no stratum, on any dataset or metric, shows Essence significantly ahead. If clustering were adding value specifically by separating genuinely distinct interests, the users most likely to show that value are exactly the ones for whom it does not appear. This does not mean clustering contributes nothing — the active cluster selection ablation (Section 5.5) shows recency-aware selection matters — but it does mean the clustering step itself, isolated from recency weighting, is not empirically supported as the source of Essence's advantage where an advantage exists.

A separate, small-scale pilot (200 users, MovieLens-25M) testing whether adaptive per-channel weighting between two embedding channels could improve on Recency-Weighted found no directional win and remains unvalidated exploratory work, not reported here as a result.

6 Discussion

6.1 Mechanism: Why Popularity-Driven Discovery Self-Reinforces

Collaborative filtering's reliance on cross-user co-occurrence creates a feedback property worth stating plainly: items that already have more interactions generate more training signal, which improves their representation in the model, which increases their likelihood of being recommended, which generates further interactions. Items with few interactions do not benefit from this loop, regardless of how well they might match any individual user's taste, because the signal needed to learn that match does not exist in sufficient quantity. This mechanism is clearest for the non-personalized popularity baseline, which records exactly zero long-tail recall on all three datasets (Table 4). Item-based CF records zero long-tail recall on the sparsest dataset, Last.fm-1K, under deterministic ranking; on the denser Amazon Books it surfaces some low-interaction items but pays a large cost to overall recall (0.0031, near the bottom of the field). Essence's retrieval step does not depend on this loop, since item representations are derived from content rather than interaction frequency: an item's likelihood of being recommended is a function of embedding similarity to the active cluster, independent of how many other users have interacted with it. MovieLens-25M is the case where this loop runs at full strength in the other direction: CF's Recall@10 (0.0850) is the highest value recorded anywhere in this study, driven almost entirely by a small set of densely-rated popular titles, and Popularity itself ranks 3rd of 10 systems overall (0.0517) — yet even here, CF's LT-Recall@10 (0.0037) is far below its own overall Recall@10, confirming that strong aggregate performance from co-occurrence signal does not translate into long-tail recovery; it is domain conditions, not a fixed property of collaborative filtering, that determine how strong this loop runs (Section 5.2).

6.2 Deployment Positioning: A Candidate-Generation Method, Not a Ranking Replacement — and Not a Universal One

The absolute recall values reported in Section 5 (approximately 0.005–0.03 under full-catalog evaluation) should be interpreted in light of how such a system would actually be deployed. Essence is not proposed as a replacement for a primary, CF-driven recommendation feed; CF baselines remain effective at surfacing broadly popular content efficiently, which is appropriate for a general homepage or default feed. On Amazon Books and Last.fm-1K, Essence is better understood as a specialized retrieval path: a candidate-generation method suited to a dedicated discovery surface (for example, a "more like this" or deep-catalog-exploration interface) where the explicit goal is surfacing items dissimilar from what population-level signals would otherwise recommend.

This positioning does not extend cleanly to every domain, and MovieLens-25M is the counterexample: where collaborative and popularity signal is strong, Essence trails not only CF and Popularity but both neural multi-interest baselines as well, by the largest effect sizes observed in this study (Section 5.2). A deployment decision should be conditioned on the domain's collaborative signal strength: whether Essence is a good candidate-generation method is predictable from measurable properties of the domain, rather than a fixed property of peer-isolated, content-only personalization.

6.3 Privacy and Deployment Constraints

Because Essence consults no cross-user data at any stage, it is deployable in settings where collaborative filtering is not viable: privacy-regulated environments, on-device recommendation without server-side interaction logs, federated settings where data cannot leave the user's device, and cold-start contexts where a population-level interaction graph does not yet exist. This is a direct consequence of the architecture (Section 3) rather than an added feature, and follows

from the same data-level distinction discussed in Section 2.2.

7 Limitations and Future Work

- **Scale.** Last.fm-1K evaluation uses 99 users. Amazon Books and MovieLens-25M each replicate on 2,000 users, but larger-scale evaluation across all three datasets would strengthen generalisability claims. In particular, the Last.fm-1K long-tail comparisons rest on few eligible long-tail hits (as few as 2 of 92 users per method) and are directional rather than statistically substantive (Section 5.7); the Amazon Books and MovieLens-25M results, at 2,000 users each, carry the statistical weight.

- K **sensitivity.** $K = 3$ is used throughout. An ablation over $K \in \{2, 3, 4, 5\}$ (Section 5.7) shows $K = 3$ is clearly suboptimal on Last.fm-1K but close to optimal on Amazon Books; this sweep has not been run on MovieLens-25M, so whether $K = 3$'s weak performance there (Section 5.2) is partly a K-choice artifact rather than purely a domain effect remains untested.

- **Active cluster selection.** The recency-based heuristic is simple by design. A learned selection mechanism could improve session-aware accuracy without requiring cross-user training.

- **Filter bubble risk.** Peer-isolation removes the serendipity that cross-user signals can provide. Whether repeated centroid queries cause semantic narrowing is an open empirical question.

- **Embedding input curation.** Section 5.6 demonstrates that noisy embedding inputs degrade performance. The quality of what enters the embedding determines the quality of what Essence can find. Automated curation of embedding input text (filtering off-topic or irrelevant content before encoding) is a direct practical extension.

- **Audio-based embeddings.** Current embeddings derive from item metadata text only. For music, incorporating audio-based representations (capturing sonic texture, timbre, rhythm, and harmonic structure directly from audio signal) would more precisely model the latent acoustic fingerprint underlying individual taste. This represents the primary planned extension of Essence: replacing or augmenting text embeddings with audio embeddings and measuring whether long-tail recall improves when the embedding captures what the music actually sounds like rather than what it is called.

- **Cross-space embedding alignment.** Pass 2 results suggest that aligning review-text and metadata input distributions within the shared embedding space (via contrastive training or a learned projection layer) could unlock the full signal of user-authored representations while maintaining retrieval coherence.

- **Hand-implemented, not pre-trained, multi-interest baselines.** MIND and ComiRec are evaluated here (Section 4) via from-scratch PyTorch implementations trained per dataset, since neither is available in standard recommendation libraries, rather than via pre-trained, production-tuned checkpoints. A well-tuned, larger-scale MIND or ComiRec deployment could plausibly outperform the versions evaluated here, particularly on MovieLens-25M, where both already outrank Essence substantially.

- **The domain-dependence mechanism is not fully explained.** "Collaborative/popularity signal strength" (Section 5.2) is measured post hoc via the baselines' own performance in each domain, not from an independent domain property decided in advance. A more principled, dataset-independent characterization of when peer-isolated personalization should be expected to help is left to future work.

8 Conclusion

We presented Essence, a personal embedding cluster framework that generates recommendations by clustering a user's item history into K semantic centroids and retrieving items by similarity to the centroid closest to the user's recent activity, without consulting any cross-user data. We evaluated Essence on three datasets spanning music, e-commerce, and film (Last.fm-1K, Amazon Books, MovieLens-25M) against nine baselines: a non-personalized popularity baseline, a real collaborative-filtering baseline, a content-based average-embedding baseline, three recency-based baselines that use the same embeddings as Essence but no clustering, and two hand-implemented multi-interest neural baselines (MIND, ComiRec).

Essence's performance relative to these baselines is domain-dependent, and the dependence has a specific, testable shape: Essence's rank among the ten evaluated systems tracks inversely with the strength of collaborative/popularity signal available in the domain (Section 5.2). On Amazon Books and Last.fm-1K, where that signal is structurally weak, Essence sits in the upper half of the field, significantly beating collaborative filtering, popularity, content-averaging, and both neural baselines on Recall@10 or LT-Recall@10 in at least one dataset — but losing to simple recency-based retrieval that uses the identical embeddings without any clustering step. On MovieLens-25M, where collaborative signal is strong, Essence ranks 8th of 10 systems, losing to collaborative filtering, popularity, and both neural baselines by the largest effect sizes observed anywhere in this study.

Whether peer-isolated, content-only personalization helps is predictable in advance from how strong collaborative and popularity signal already is in a given domain. A targeted stratification test (Section 5.7) finds no evidence that the K-means clustering step, isolated from recency-weighted retrieval, is itself the source of Essence's advantage where an advantage exists. Essence is a data point on when peer-isolated, content-only personalization is and isn't the right architectural choice, not a universally superior candidate generator.

An ablation (Section 5.5) indicates that Essence's long-tail advantage, where present, depends on its recency-based active cluster selection mechanism specifically, not on K-means clustering alone: replacing recency-based selection with a static mean-embedding query degrades performance below the content baseline. This finding has been verified on Last.fm-1K only, and we have not yet confirmed whether it holds on Amazon Books or MovieLens-25M.

The long-tail definition used here (items with exactly one training-set interaction) was applied independently, without modification to the threshold or its derivation, to three datasets spanning different domains, scales, and interaction densities. The resulting singleton populations differ substantially in where they sit within the catalog — concentrated at the rare-popularity end on Last.fm-1K and Amazon Books (deciles 3–9), but more centrally on MovieLens-25M (deciles 2–5, Section 4) — a dataset-specific difference in decile geometry that the cold-start analysis (Section 5.4) shows matters for interpreting long-tail results correctly, not a failure of the definition itself.

Limitations of the present evaluation, beyond those above, are discussed in Section 7, including dataset scale, sensitivity to the choice of K, and the increased recall degradation observed when taste profiles are built from noisy, user-authored text (Section 5.6) rather than structured item metadata.

References

[1] Li, C., Liu, Z., Wu, M., Xu, Y., Zhao, H., Huang, P., Kang, G., Chen, Q., Li, W., & Lee, D. L. (2019). *Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.* CIKM 2019.

[2] Cen, Y., et al. (2020). *Controllable Multi-Interest Framework for Recommendation*. KDD 2020.

[3] Sun, F., et al. (2019). *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer*. CIKM 2019.

[4] He, X., et al. (2017). *Neural Collaborative Filtering*. WWW 2017.

[5] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer.

[6] Hidasi, B., et al. (2016). *Session-Based Recommendations with Recurrent Neural Networks*. ICLR 2016.

[7] Wang, W., et al. (2020). *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. NeurIPS 2020.

[8] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP 2019.

[9] Celma, O. (2010). *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. Springer.

[10] Hou, Y., et al. (2024). *Bridging Language and Items for Retrieval and Recommendation*. arXiv:2403.03952.

[11] Krichene, W., & Rendle, S. (2020). *On Sampled Metrics for Item Recommendation*. KDD 2020.

[12] Ferrari Dacrema, M., Cremonesi, P., & Jannach, D. (2019). *Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches*. RecSys 2019.

[13] Ferrari Dacrema, M., Boglio, S., Cremonesi, P., & Jannach, D. (2021). *A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research*. ACM Transactions on Information Systems.